Scaleform GFx

AMP User Guide

This document describes the Scaleform Analyzer for Memory and Performance (AMP) and how to use the system for profiling applications.

Author: Alex Mantzaris
Version: 2.06
Last Edited: August 16, 2011

Copyright Notice

Autodesk® Scaleform® 4

© 2011 Autodesk, Inc. All rights reserved. Except as otherwise permitted by Autodesk, Inc., this publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

Certain materials included in this publication are reprinted with the permission of the copyright holder.

The following are registered trademarks or trademarks of Autodesk, Inc., and/or its subsidiaries and/or affiliates in the USA and/or other countries: 3DEC (design/logo), 3December, 3December.com, 3ds Max, Algor, Alias, Alias (swirl design/logo), AliasStudio, Alias|Wavefront (design/logo), ATC, AUGI, AutoCAD, AutoCAD Learning Assistance, AutoCAD LT, AutoCAD Simulator, AutoCAD SQL Extension, AutoCAD SQL Interface, Autodesk, Autodesk Intent, Autodesk Inventor, Autodesk MapGuide, Autodesk Streamline, AutoLISP, AutoSnap, AutoSketch, AutoTrack, Backburner, Backdraft, Beast, Built with ObjectARX (logo), Burn, Buzzsaw, CAiCE, Civil 3D, Cleaner, Cleaner Central, ClearScale, Colour Warper, Combustion, Communication Specification, Constructware, Content Explorer, Dancing Baby (image), DesignCenter, Design Doctor, Designer's Toolkit, DesignKids, DesignProf, DesignServer, DesignStudio, Design Web Format, Discreet, DWF, DWG, DWG (logo), DWG Extreme, DWG TrueConvert, DWG TrueView, DXF, Ecotect, Exposure, Extending the Design Team, Face Robot, FBX, Fempro, Fire, Flame, Flare, Flint, FMDesktop, Freewheel, GDX Driver, Green Building Studio, Heads-up Design, Heidi, HumanIK, IDEA Server, i-drop, Illuminate Labs AB (design/logo), ImageModeler, iMOUT, Incinerator, Inferno, Inventor, Inventor LT, Kynapse, Kynogon, LandXplorer, LiquidLight, LiquidLight (design/logo), Lustre, MatchMover, Maya, Mechanical Desktop, Moldflow, Moldflow Plastics Advisers, MPI, Moldflow Plastics Insight, Moldflow Plastics Xpert, Moondust, MotionBuilder, Movimento, MPA, MPA (design/logo), MPX, MPX (design/logo), Mudbox, Multi-Master Editing, Navisworks, ObjectARX, ObjectDBX, Opticore, Pipeplus, PolarSnap, PortfolioWall, Powered with Autodesk Technology, Productstream, ProMaterials, RasterDWG, RealDWG, Real-time Roto, Recognize, Render Queue, Retimer, Reveal, Revit, RiverCAD, Robot, Scaleform, Showcase, Show Me, ShowMotion, SketchBook, Smoke, Softimage, Softimage|XSI (design/logo), Sparks, SteeringWheels, Stitcher, Stone, StormNET, StudioTools, ToolClip, Topobase, Toxik, TrustedDWG, U-Vis, ViewCube, Visual, Visual LISP, Volo, Vtour, WaterNetworks, Wire, Wiretap, WiretapCentral, XSI.

All other brand names, product names or trademarks belong to their respective holders.

Disclaimer

THIS PUBLICATION AND THE INFORMATION CONTAINED HEREIN IS MADE AVAILABLE BY AUTODESK, INC. “AS IS.” AUTODESK, INC. DISCLAIMS ALL WARRANTIES, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE REGARDING THESE MATERIALS.

How to Contact Autodesk Scaleform:

Document	AMP User Guide
Address	Autodesk Scaleform Corporation 6305 Ivy Lane, Suite 310 Greenbelt, MD 20770, USA
Website	www.scaleform.com
Email	info@scaleform.com
Direct	(301) 446-3200
Fax	(301) 446-3199

Table of Contents

1 Introduction 1

1.1 AMP Server 2

1.2 AMP Client 3

2 Getting Started with AMP 8

2.1 Making a Connection 8

2.2 Analyzing Performance Metrics 9

2.3 Analyzing Rendering Metrics 10

2.4 Analyzing Memory Consumption 10

2.5 Using Overdraw Profiling 14

2.6 Using Batch Profiling 15

3 Platform and Integration Notes 16

4 Build Notes 17

5 Adding AMP Support 18

5.1 Debug File Generation 18

5.2 Application Control 18

5.3 Connection Status 19

5.4 Markers 19

6 FAQs Related to AMP 21

7 Additional Information 22

1 Introduction

AMP™ is the Scaleform® remote profiling system, capable of monitoring CPU usage, graphics rendering, and memory consumption of a Scaleform application. Real-time graphs provide a method of quickly identifying memory and performance bottlenecks, and per-frame statistics allow an in-depth analysis and determination of the causes. AMP can also help find script problem areas, using its per-frame ActionScript function and source code timings.

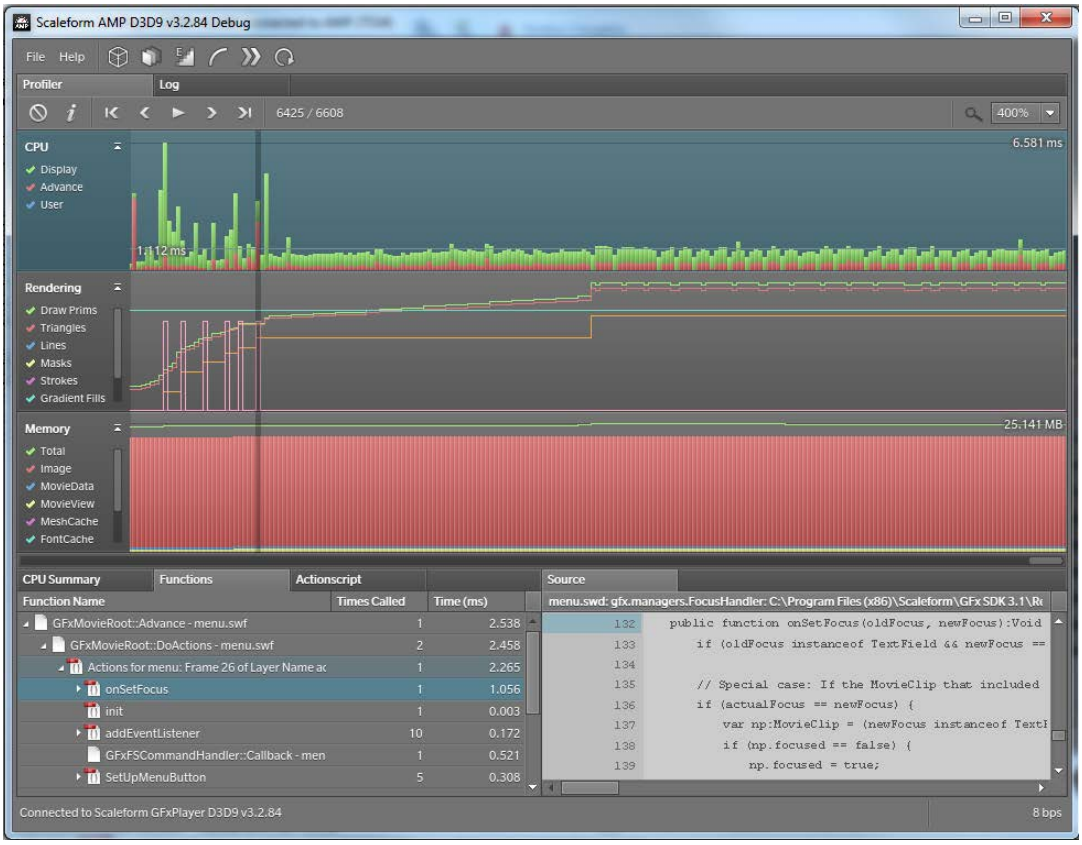


Figure 1: Scaleform AMP

Every Scaleform application can act as an AMP server, which means that it listens for incoming connection requests by the profiler, gathers information every frame, and sends it to the profiler over the network. The profiler acts as an AMP client, which means that it connects to a specific AMP server, processes the information sent from the server every frame, and sends requests to control the profiled application.

AMP is available with Scaleform 3.2 and higher versions.

1.1 AMP Server

A Scaleform application being profiled (the AMP server) sends a certain amount of information over the network to the profiler. Depending on the level of function profiling, each ActionScript function and some important C++ functions may be timed. If per-line source code profiling is enabled in ActionScript 2, each ActionScript bytecode execution is timed (ActionScript 3 source code profiling does not require timing each execution). In addition, rendering statistics, detailed memory heap and image information, flash file information, and ActionScript markers are recorded. Note that this extra work per frame can slow down the application and increase its memory consumption, especially if per-line timings are enabled.

In order to keep AMP from interfering with the application being profiled, all AMP server information is kept on a separate memory heap. If the size of this heap exceeds a pre-set limit, the application is paused until the pending frame information is sent over the network and subsequently deleted. Scaleform Shipping builds do not act as AMP servers, and AMP support may be easily disabled for any build within an application, if so desired.

Performance statistics on the server are maintained per Movie. This allows users to easily identify problem spots in the case where there are several views associated with a particular SWF file. It also allows AMP to produce meaningful call graphs for ActionScript execution of multiple Movies on separate threads.

In addition to reporting memory and statistics to the client, the server can be controlled by the AMP client in several ways to assist profiling. For example, the profiler can send requests for wireframe mode rendering, for pausing, fast-forwarding, or restarting the flash movie, for toggling the anti-aliasing and stroke modes, for changing the localization fonts, and for changing the vector graphics' curve tolerance. Furthermore, the client can toggle special AMP rendering modes to show display performance bottlenecks, such as mask, filter, and pixel overdraw highlighting.

While all the above functionality has been implemented in Scaleform Player, some of it requires application-specific implementation, and may be performed as part of a Scaleform integration. To graciously handle the case where some of these capabilities have not been implemented in a Scaleform application, the AMP server reports the set of supported application control functionality to the client on connection. It also reports its current state (wireframe mode, for example) to the client for visual feedback.

Finally, the AMP server has the ability to send Flash debug information over the network to the profiler, so that source code and per-line timings can be displayed if the client does not have local access to that information. In order for this transfer to occur, the SWD file (ActionScript2) or AS file (ActionScript3) should be in a location where AMP can find it.

1.2 AMP Client

The remote profiler (AMP client) is a stand-alone application that connects to an AMP server, receives profile information every frame, and displays this information in a user-friendly way to enable effective analysis of performance and memory bottlenecks. The profiler is itself a Scaleform application, implemented using Scaleform CLIK.

The AMP client has several features that allow developers to view multiple aspects of their Flash assets and identify problem areas:

- **CPU graph**: Shows how much time was spent per game frame in ActionScript (Advance), Scaleform rendering (Display), and in the Direct Access API (User). This graph is especially useful to identify spikes in CPU usage due to Scaleform. When a frame with a spike is selected, the cause can be identified by examining the detailed statistics for that frame. When the CPU graph is selected, the following tabs become visible in the bottom panel of the application:
    - **CPU Summary:** Breaks down the Advance, Display, and User times into sub-categories to provide a more in-depth overview of where the time was spent for the selected frame(s). If multiple frames are selected, then the values will be the average over those frames.
    - **Functions:** Shows a tree control with the amount of time spent inside each function, inclusive of the functions that were called from it. When multiple frames are selected the time and number of calls are an average per game frame. ActionScript and C++ functions are mixed in the same call graph because the code path may move between the ActionScript VM and the rest of the application. Expanding each function reveals the functions that were called from that function.
    - **Actionscript:** This tab shows similar information as the Functions tab, but the information is not hierarchical, C++ functions are excluded, and the time for each function excludes the time spent in other functions in the list. The tab is useful to quickly identify the most expensive script functions.
    - **Source:** Shows the amount of time spent in each line of the code for the selected function in the Functions tab. For AS3, per-line timings are always enabled. For AS2, in order to see per-line timings, which are off by default, the “high” level of function profiling needs to be selected from the drop-down control on the toolbar. Instruction profiling in AS2 is time-consuming, and can slow down the Scaleform application being profiled, so it is best done after a problem area has already been identified. The timings shown next to each line do not add up to the amount of time reported in the function call graph in the case where the function was called by more than one other function. This is because the per-line timings are the total time spent in the function regardless of from where it was called. Also, line timings do not include the time spent inside function calls, whereas the call graph timings include time spent in called functions. In order to display source code and per-line timings, any flash debug files *not found on the AMP*

server (SWF location) need to be in the working directory of the AMP client executable, or in the case of AS3 source files, in the same relative path to the loaded SWF.

- **Rendering graph**: The Scaleform Best Practices Guide document identifies several rendering-related areas that may cause performance hits, such as an excessive number of draw primitives, triangles, and lines, or the presence of masks, filters, strokes, or gradient fills. Counters for each of these areas are shown in the rendering graph, so that developers can gain more insight to what is happening inside Scaleform. When the Rendering graph is selected, the **Render Summary** tab becomes visible at the bottom panel of the application, which shows the same information as the graph, but in numeric form. This tab also includes some additional rendering statistics, such as mesh and font cache thrashing (number of items evicted during the frame), font fill rate (the area of all the glyphs as a percentage of the total area of the font cache textures), font cache failure rate (how many glyphs failed to allocate during the frame, and were replaced by vector shapes), number of font cache texture updates during the frame, and current number of font textures.

- **Memory graph:** The memory consumption of Scaleform is tracked in the memory graph, allowing the developer to quickly monitor memory usage by category such as rendering, images, sound, fonts, movie data, etc, and includes all loaded files. When the Memory graph is selected, the following tabs become visible at the bottom panel of the application:
    - **Memory:** This tab allows the user to determine in detail the Scaleform footprint, how much memory claimed by Scaleform is actually in use, and how memory is allocated by purpose. When profiling in full memory report mode, which is toggled by the "i" button on the AMP toolbar, the memory report becomes more detailed, showing a breakdown by heap, by file, and by purpose.
    - **Images:** Shows the memory consumed by each image, along with its dimensions and format. Clicking on each item in the list displays a thumbnail of that image in the preview pane. Note that the image data are not sent as part of the profiling information, but are sent upon request. Therefore, it is possible that no image is displayed if that image has already been unloaded by Scaleform.
    - **Fonts:** Shows the total memory consumed by the font cache, as well as the font list for each SWF.

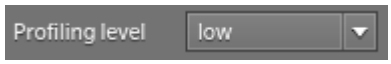
- **File Menu: T**his menu includes the following choices:
    - **Connection:** Brings up the connection dialog, for connection to a different server.
    - **Disconnect:** Terminates the current connection.
    - **Add Path:** Brings up a "select folder" dialog, which allows the user to select additional folders to be added to the list of locations where AMP looks for SWD files (for AS2) or AS files (for AS3).
    - **Clear Paths:** Clears the list of locations where AMP looks for SWD and AS files.
    - **Load Profile Frames:** Loads a previously-saved profile run for reexamination. Selecting this menu item brings up a file load dialog, which is used to locate the profile data file.

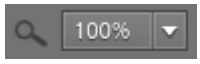
- **Save Profile Frames:** Saves the current profile run to disk. The run can be reloaded by selecting the "Load Profile Frames" option, above. Selecting this menu item brings up a file save dialog, which is used to select the path and name of the file containing the profile data.
- **Exit:** Closes the application.

- **Help Menu:** This menu currently has two options: "User Guide," which links to this document in the Scaleform.com developer center, and "About," which brings up a screen with the AMP version and credits.

- **Application Control Toolbar:** This UI element contains a series of buttons for controlling the application being profiled.
  -  **Wireframe:** Sends a request to the profiled application to render content in wireframe mode. When the application is already in wireframe mode, clicking the button takes it out of that mode.
  -  **Overdraw mode:** Sends a request to the profiled application to enter a rendering mode that highlights areas of pixel overdraw, masks, and filters. The AMP server renders areas of pixel overdraw with green (color intensity is proportional to number of overdraws), masks with red, and filters with blue. When the application is already in overdraw mode, clicking the button takes it out of that mode.
  -  **Batch profiling mode:** Sends a request to the profiled application to ender a rendering mode that draws each render batch in a different color. When the application is already in batch profiling mode, clicking the button takes it out of that mode.
  -  **Anti-aliasing:** Sends a request to the profiled application to cycle through anti-aliasing modes. The three modes available are Scaleform's edge anti-aliasing technique (EdgeAA), Hardware- full-scene anti-aliasing (HW FSAA), or no anti-aliasing (None).
  -  **Stroke:** Sends a request to the profile application to cycle through stroke modes. The three modes available are Normal, Correct, and Hairline.
  -  **Fast-forward:** Sends a request to the profiled application to play the flash content in fast-forward. This causes the SWFs to advance as fast as the rendering frames and disregard its document FPS setting. When the application is already in fast-forward mode, clicking the button takes it out of that mode.
  -  **Restart:** Sends a request to the profiled application to restart the flash content.

- **Profiling Toolbar:** This element consists of a series of buttons that are used for profiling.

-  **Clear:** Discards the current profile run. The profile frames are discarded, thereby freeing up AMP client memory for new data.
-  **Detailed Profiling** (detailed memory breakdown): When this option is selected, AMP gathers more detailed memory statistics per frame, which are displayed in the memory tab. This option also causes AMP to display the full list of images in the Images tab, rather than a maximum of ten.
-  **First Frame:** Selects the first frame in the current profile run, thereby updating the information displayed on the bottom panel tabs.
-  **Previous Frame:** Selects the previous frame from the currently-selected one, thereby updating the information displayed on the bottom panel tabs.
-  **Pause/Play:** Sends a request to the profiled application to stop sending memory and performance data. This operation does not pause the application itself. Clicking the button again resumes the profiling.
-  **Next Frame:** Selects the next frame from the currently-selected one, thereby updating the information displayed on the bottom panel tabs.
-  **Last Frame:** Selects and locks onto the last frame in the current profile run. As information for new frames is received, the selection is changed to always be the last received frame, and the information displayed on the bottom panel tabs is continuously updated.
- 

Profiling Level: This drop-down control allows three levels of function profiling: “Low,” “Medium,” and “High.” A low level of profiling causes AMP to time only some high-level C++ functions. A medium level of profiling results in ActionScript functions being timed as well as the functions from the previous level. A high level of profiling also gathers data from some lower-level C++ functions. In addition, the high level of profiling keeps track of the time spent in each line of ActionScript 2 code, which is displayed in the Code panel for the selected frame. A debug file (SWD) is required in order to show source line timings. In the case of ActionScript 3, these timings are always shown.
- 

Zoom: This drop-down control on the far-right alters the horizontal scale of the graphs. Zooming out allows identification of problem areas of the run, and zooming in allows more precise examination of those areas. The mouse wheel can also be used for zooming in and out, and provides a higher resolution not available via the drop-down.

- **Log:** Clicking on the Log tab displays the Scaleform log that has been produced by the profiled application over the course of the connection. Clicking back on the Profile Tab brings back the display with the memory and performance graphs and detail tabs.

- **Status:** At the bottom of the profiler is a Status Bar that shows the current connection and bit rate of the received information over the network.

2 Getting Started with AMP

To start using the AMP profiler, run the *GFxAmpClient.exe* executable that is located in Bin/AMP under the main Scaleform installation directory. The application starts, and the connection dialog appears.

2.1 Making a Connection

A connection dialog appears automatically when the client starts up. It prompts the user to either select from a list of discovered AMP servers, or manually type the IP address and port to connect. Select the second approach if the AMP server of interest is not yet running, as would be the case if there is a need to profile an application as it is starting up.

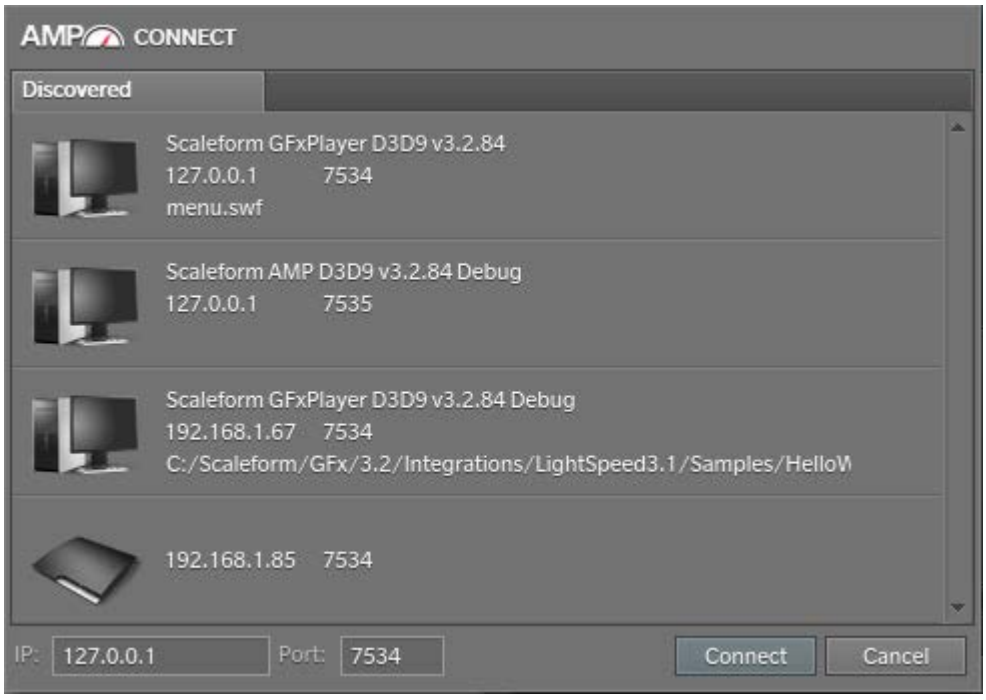


Figure 2 : AMP connection dialog

The Connection item under the File menu can be used to bring up the connection dialog at any time. Each AMP server in the discovered list has a distinct icon identifying its platform (PC, Xbox360, PS3, etc). Next to the icon is the title of the application, the IP address and port where the server is listening, and the Flash content currently being played. If some of this information is missing, it is because the corresponding AMP server is not sending it. A description of how to add full AMP support to any Scaleform application, including broadcasting the application and content information, is found later in

this document. A connection is established either by double-clicking a discovered server, or by selecting a server and clicking the “Connect” button.

Please note that connection to a Wii development kit (NDEV) does not use the above server discovery method. Instead, the Wii is physically connected to a PC using the USB interface (COM port) built into the NDEV. Connect to a Wii AMP server from the connection dialog by specifying “wii” in the IP address field.

2.2 Analyzing Performance Metrics

Once a connection has been established, profiling information begins to flow into the client, and the graphs start getting populated. The top graph shows how much CPU time is spent per frame rendering Scaleform (Display), executing ActionScript (Advance), and in Direct Access API (User).

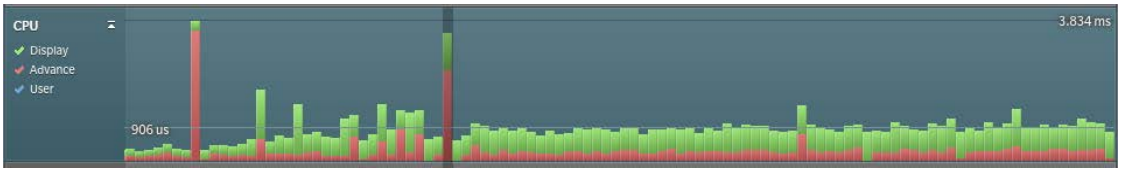


Figure 3: CPU Graph

If the CPU graph is not of interest at some point during the analysis, it may be minimized so that more room is available for other UI elements. If one of the graphed quantities is not of interest, it may be removed by clicking on the checkmark next to its legend.

With a frame on the CPU graph selected, the CPU Summary, Functions, and ActionScript tabs become visible at the bottom of the profiler. The CPU Summary tab shows the graph information in more detail, the Functions tab shows the call graph for the selected frame, and the ActionScript tab shows a non-hierarchical view of all script functions called during the selected frame.

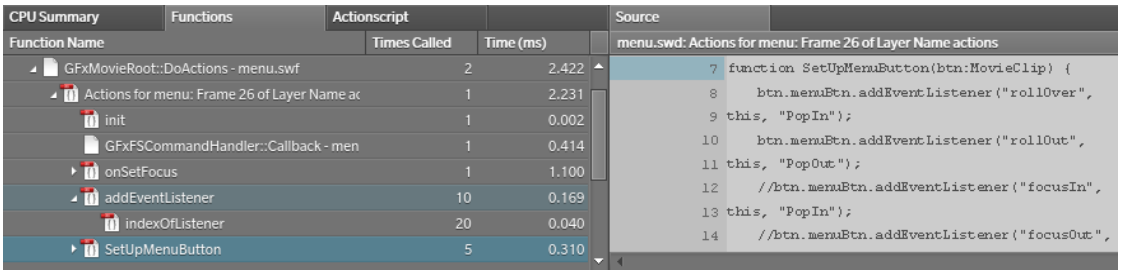


Figure 4 : CPU Summary

By examining the CPU detail tabs for a frame with a spike in the CPU graphs, the cause of script performance bottlenecks may be identified. On the other hand, rendering performance bottlenecks are more easily found by examining the rendering graph.

2.3 Analyzing Rendering Metrics

The rendering graph shows number of draw primitives, number of triangles, number of lines, number of masks used, number of strokes, number of gradient fills, number of glyphs in the mesh cache, and number of font updates that occurred during rendering for the selected frame. The Best Practices Guide explains in detail why minimizing the above quantities is important for performance.

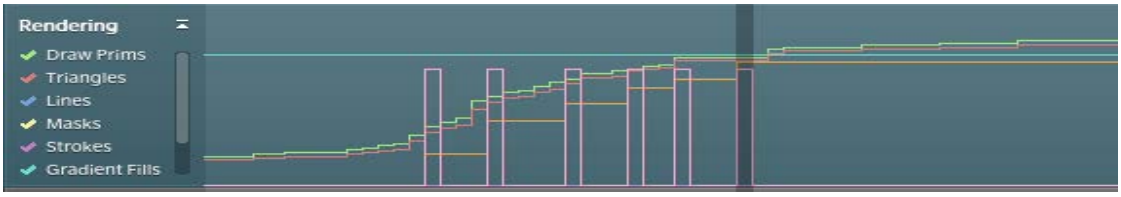


Figure 5: Rendering Graph

When a frame on the rendering graph is selected, the Render Summary tab becomes visible at the bottom of the application.

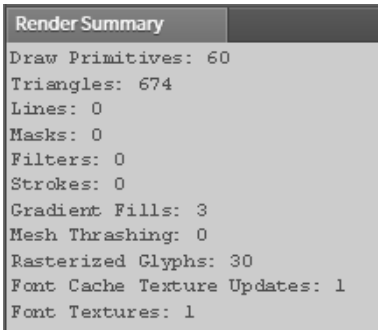


Figure 6: Rendering Summary

2.4 Analyzing Memory Consumption

The third graph from the top shows Scaleform memory usage:

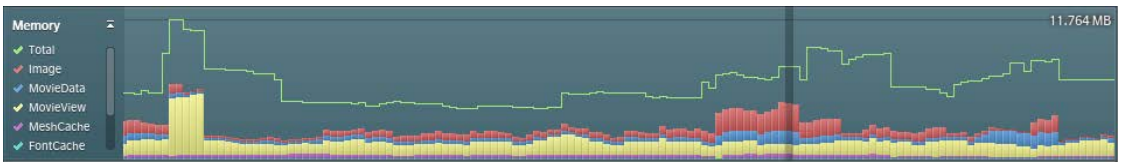


Figure 7: Memory graph

- **Total:** The total memory claimed by Scaleform (footprint). The total memory is drawn as a line graph, and includes all the categories listed below, as well as unused memory due to overhead, granularity in requested allocations, and fragmentation. Note that the total memory is greater

than the sum of the used memory categories due to unused memory not being displayed in any category.

- **Image:** Memory used for images (loaded DDS textures). Since Scaleform 4.0 keeps images in video memory, not RAM, this category is usually small compared to Scaleform 3.x.
- **MovieData:** Memory occupied by all loaded SWF or GFX files. Object data, embedded fonts, and timeline animations affect this number.
- **MovieView**: Runtime memory occupied by GFx::Movie instances, allocated for timeline animation support, on-screen objects, and ActionScript.
- **MeshCache:** Memory occupied for the cached shape mesh data, generated by the vector tesselator and edge anti-aliasing
- **Font Cache:** Memory used by the font cache.
- **Sound**: Memory used for embedded sound data. It is affected by the number, length, or quality of embedded sound samples.
- **Video:** Memory used for video memory playback buffers. These buffers are allocated when videos start, and released when they stop.
- **Other:** Memory that is in use by Scaleform, but not part of the above categories.

See the Memory System Overview document for more details on the memory system and how to configure, optimize, and manage it.

When the memory graph is selected, the Memory, Heaps, Images, and Fonts tabs become visible at the bottom of the application. The Memory tab shows the above memory categories in more detail. All values in the memory tabs are in bytes, unless noted otherwise.



Figure 8: Memory

Expanding each category reveals subcategories or memory heaps responsible for direct allocations in that category, if any. Drilling down in this manner, it is possible to gain a deeper understanding of Scaleform memory use in any particular frame.

The next Memory tab displays the images loaded in the selected frame(s), along with their sizes.

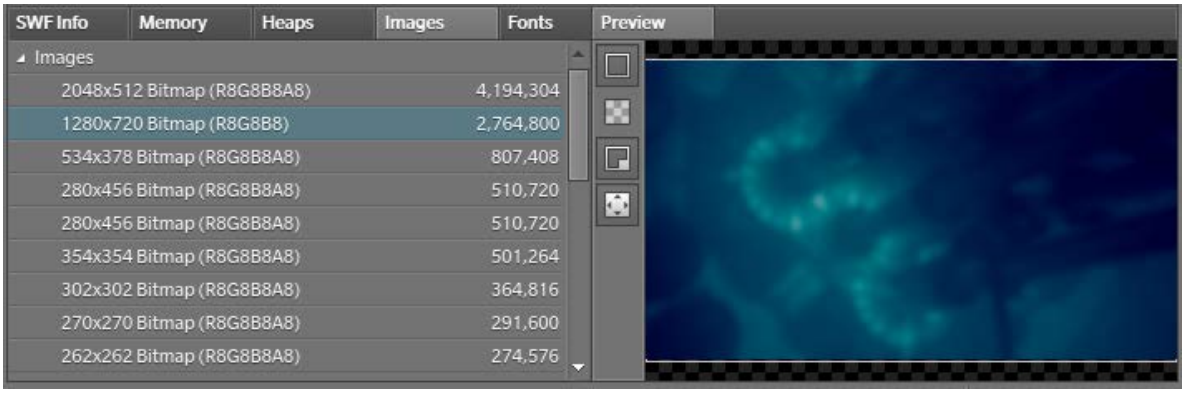


Figure 99: Images memory tab

Use this tab to quickly check whether a large image is present that increases the memory consumption of the target Scaleform application. Clicking on an item in the image list displays a thumbnail of that image in the preview pane. Note that the image data are not sent as part of the

profiling information, but are sent upon request. Therefore, it is possible that no image is displayed if that image has already been unloaded by the connected Scaleform application.

The final Memory Tab displays font information for the selected frame(s).

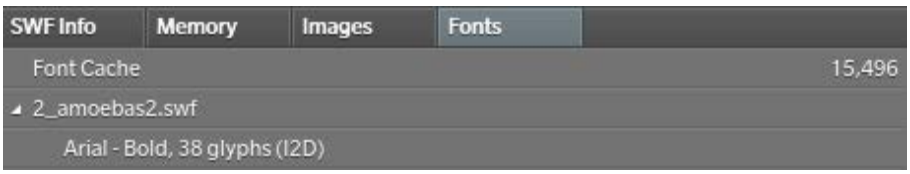


Figure 10: Fonts memory tab

Use this tab to examine the number of embedded glyphs for each font type, per SWF file.

Embedded fonts can consume a significant amount of memory, which shows up in AMP by increased sizes of the MovieData heap and the CharDefs category. These may be examined in the Heaps and Memory tabs, respectively. Font memory may be reduced by sharing fonts over several SWF files.

2.5 Using Overdraw Profiling

AMP may be used to remotely trigger a special rendering mode that highlights potential problem areas in the profiled application. To toggle this rendering mode, click on the appropriate button located on the application control toolbar.

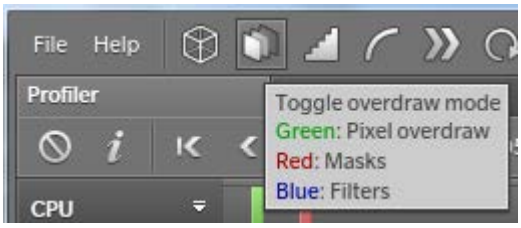


Figure 11: Toggle Button

Once this mode is toggled, all objects are rendered in green. Green indicates pixel overdraw (16 layers for full brightness), masks are red, and filters are blue (8 layers of masks or filters for full brightness). These colors blend together, so a bright yellow area is high overdraw with nested masks.

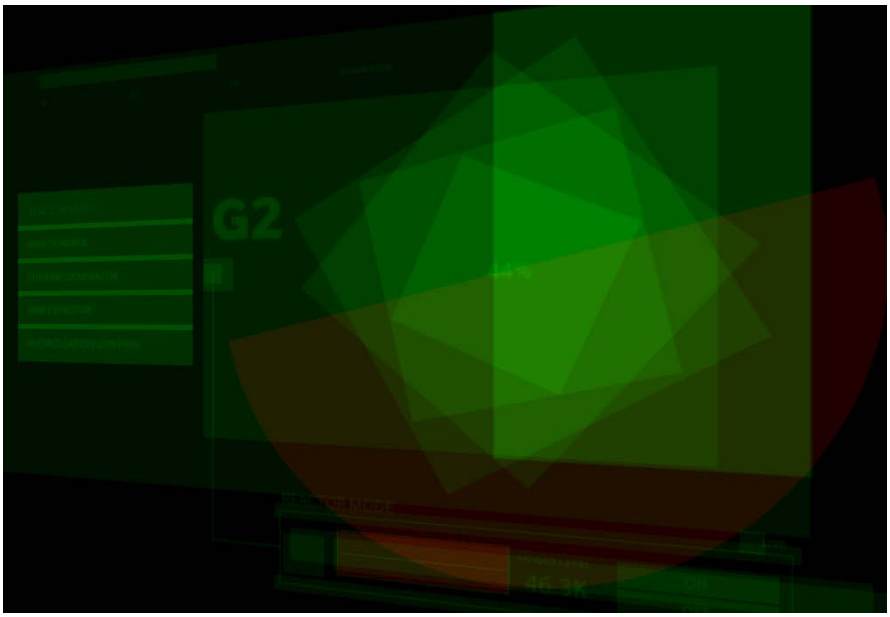


Figure 12: Rendering mode for Overdraw profiling

Overdraw, masks, and filters all affect performance, and should be minimized whenever possible.

2.6 Using Batch Profiling

Another special rendering mode that AMP supports is batch profiling. To toggle this rendering mode, click on the appropriate button located on the application control toolbar.

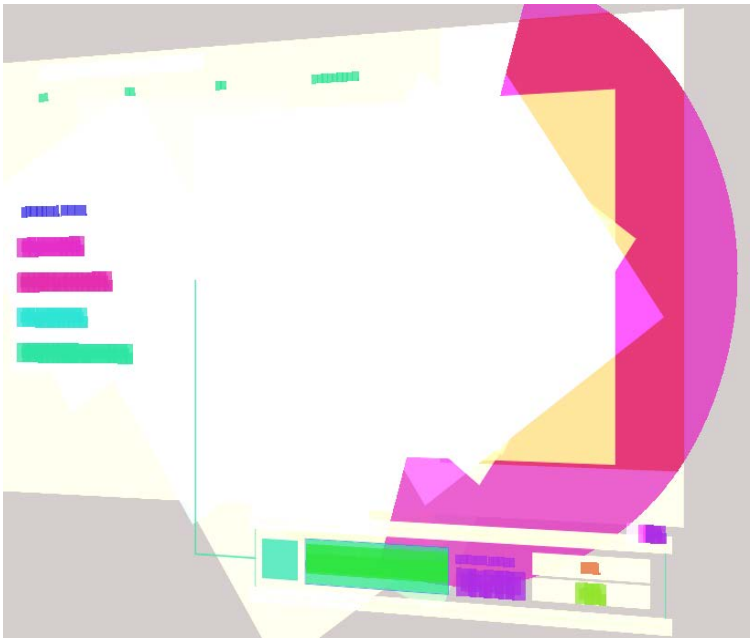


Figure 13: Batch profiling mode

Once this mode is toggled, each rendering batch is rendered in a different color. Best performance is achieved by minimizing the number of batches, whenever possible.

3 Platform and Integration Notes

1. AMP needs to receive information every game frame in order to display memory and performance statistics. The following call should be inserted in the rendering thread to explicitly update AMP: `AmpServer::GetInstance().AdvanceFrame()`

2. The AMP client application is currently available only for Windows.

3. AMP server is currently supported for Windows, Xbox360, PS3, Linux, Mac, Wii, Android, and iPhone/iPad. Using the automatic discovery feature of the AMP client, connecting is straightforward. However, connecting without automatic discovery requires knowledge of the IP address for the AMP server, which can cause confusion because some platforms have multiple IP addresses.

   - For the Xbox360, please make sure to specify the title IP address, not the debug IP address. The title IP address can be determined by viewing the properties of the development or test kit in the Xbox 360 neighborhood.
   - For the PS3, if the development kit is configured to use two IP addresses, please make sure to specify the application IP address, not the IP address used for debugging.
   - For Android, internet permission needs to be enabled at the application level. To do this, the following needs to be added to AndroidManifest.xml for the application:

     ```
     <uses-permission android:name="android.permission.INTERNET" />
     ```

   - The Wii does not use network sockets for communication with AMP. Instead, the console is physically connected to a PC using the USB interface (COM port) built into the NDEV. Users may connect to a Wii AMP server from the connection dialog by specifying “wii” in the IP address field. Important: Please make sure RVL_SDK/X86/bin is in the PATH environment variable, so that hio2.dll may be loaded by AMP.

4. Some game engines, such as Unreal Engine 3, package their assets in such a way that makes it impossible for the AMP server to locate the AS2 SWD or AS3 AS files corresponding to the flash content being profiled. If that is the case, please make sure to place the SWD or AS files in the AMP client working directory, rather than relying on the AMP server to find them in the same directory as the SWF and send them over the network.

4 Build Notes

1. AMP is part of all non-shipping Scaleform builds by default. To remove AMP from Scaleform, make sure SF_AMP_SERVER is undefined in GFxConfig.h, and rebuild the Scaleform libraries. If Scaleform source access is not available, then AMP may be disabled at run-time by calling: `AMP::Server::GetInstance().SetState(AMP_Disabled, true)`

2. AMP uses network sockets for communication with the profiler. Therefore, make sure the application links with the appropriate platform-specific network library (Ws2_32.lib for Windows, Xnet.lib for Xbox360, libnetctl for PS3, socket for Linux, etc).

3. If AMP is the only system using network sockets, it will initialize the socket library, and release it when it is done. On some platforms, releasing the library has no effect if it has been initialized more times than it has been released. In that case, all is fine. On other platforms, however, releasing the network library takes immediate effect, even if that library has been initialized more times than has been released, and AMP may interfere with socket connections used in other parts of the application. In that case, AMP may be forced to skip socket initialization and use a previously-initialized socket library by calling: `AMP::Server::GetInstance().SetInitSocketLib(false).` This call needs to be performed before AMP has already been initialized.

4. A limit may be set for the AMP memory heap. When that limit is exceeded, Scaleform is paused until any pending messages have been sent to the profiler. The limit is set by calling `AMP::Server::GetInstance().SetHeapLimit()`.

5. The AMP server will create a listening network socket on port 7534 by default. A different port may be set by calling: `AMP::Server::GetInstance().SetListeningPort().`

6. AMP can be configured to halt execution until a connection has been established with the profiler client. This is useful for obtaining statistics during startup, and is accomplished on startup by calling: `AMP::Server::GetInstance().SetConnectionWaitTime.`

5 Adding AMP Support

Any Scaleform application may be configured to support remote profiling by AMP by following the steps outlined in this section. Also, the Scaleform Player source can be examined for an example of a fully-configured AMP server.

5.1 Debug File Generation

AMP has the ability to use ActionScript 2 Flash debug information (SWD files) to display source code and per-line timings. Generate SWD files by running the debugger (Ctrl+Shift+Enter) from within Flash CS3 or CS4. Place the SWD file either in the same location as the corresponding SWF, or in the working directory of the AMP client.

For ActionScript 3, there are no SWD files, and debug information is embedded directly into the SWF. AS files are needed to display the source code. Generate debug SWF files by running the debugger from within Flash Studio.

5.2 Application Control

AMP has the ability to remotely control settings in the profiled application, such as wireframe mode, or batch profiling mode. Many of the supported settings are application-specific, and may be implemented by the application. Any such functionality not implemented will simply not be available via the AMP client.

Follow the steps outlined below to handle app-control messages sent from AMP:

- Implement a custom app-control callback class that derives from `AMP::AppControlInterface`, and override the `HandleAmpRequest` method to handle `AMP::MessageAppControl` messages from AMP.
- Install the custom handler by calling `AMP::Server::GetInstance().SetAppControlCallback` on startup.
- Inform AMP of the supported functionality by calling `AMP::Server::GetInstance().SetAppControlCaps`. The argument to this method is a `AMP::MessageAppControl` message, with the supported functionality set to true. For example:

```
AMP::MessageAppControl caps;
caps.SetCurveToleranceDown(true);
caps.SetCurveToleranceUp(true);
caps.SetNextFont(true);
```

```
caps.SetRestartMovie(true);
caps.SetToggleAaMode(true);
caps.SetToggleAmpRecording(true);
caps.SetToggleFastForward(true);
caps.SetToggleInstructionProfile(true);
caps.SetToggleOverdraw(true);
caps.SetToggleBatch(true);
caps.SetToggleStrokeType(true);
caps.SetTogglePause(true);
caps.SetToggleWireframe(true);
AMP::Server::GetInstance().SetAppControlCaps(&caps);
```

5.3 Connection Status

AMP client has the ability to display the name of the connected application in its status bar. This information is communicated by calling `AMP::Server::GetInstance().SetConnectedApp`. Also, the AMP client can display the current application state, but for this it needs to be informed whenever the application state changes. Use the following methods to update the state:

- `AMP::Server::GetInstance().SetState`, with the first argument one of the following ServerStateType enumeration types:
  - `Amp_RenderOverdraw`
  - `Amp_App_Wireframe`
  - `Amp_App_Paused`
  - `Amp_App_FastForward`
- `AMP::Server::GetInstance().SetAaMode`
- `AMP::Server::GetInstance().SetStrokeType`
- `AMP::Server::GetInstance().SetCurrentLocale`
- `AMP::Server::GetInstance().SetCurveTolerance`
- Alternatively, a `AMP::ServerState` object may be filled in with all the above information and passed to `AMP::Server::GetInstance().UpdateState`

5.4 Markers

AMP can display special indicators, markers, on the CPU graph that correspond to C++ or ActionScript calls made within the profiled application. Add a marker in ActionScript as follows:

```
Amp.addMarker(1)
```

Note: Compiling .as files that contain the Amp.addMarker() code may produce a compilation error. To avoid this, make sure to use the intrinsic Amp class provided under the Resources/AS2/CLIK directory.

Add a marker in C++ as follows:

```
GFx::MovieImpl:: AdvanceStats->AddMarker(1)
```

AMP will then display a marker on the frame where the call was made. The integer argument will be used in the future to display more than one type of marker.

6 FAQs Related to AMP

This section contains few of the questions that developers may have when using the AMP tool.

1. **What are the bare minimums I need game side to have AMP working?**

You need an AMP-enabled build (i.e. with GFX_AMP_SERVER defined). By default, all non-shipping builds of GFx have AMP enabled. Also, if your game does not call Platform::RenderThread::drawFrame, you need to make sure AMP is updated every frame, by calling AmpServer::GetInstance().AdvanceFrame

2. **When we used AmpClient to connect our game, the frame rate of our game became very slow. We found AmpServer::AdvanceFrame() takes a lot of time. Is this normal?**

As with most profilers, it is normal for there to be a performance penalty when connected to AMP. In order to minimize this performance impact, please make sure you start at a low profiling level, which can be set by selecting from the appropriate dropdown control on the AMP client toolbar. Then, once you have located the bottleneck, you may increase the profiling level at the bottleneck to determine the cause in greater detail. Requesting full memory reports at each frame may also have a significant performance impact, so please make sure these reports are produced only when you need to see them. You may toggle detailed memory reports on and off via the “i” (lowercase l) on the AMP client toolbar.

3. **I'm running the game on Windows/PS3/Xbox, and I'm able to connect with the AMP client after it finds it in the discovery window. However, all the stats are blank.**

AMP needs to receive information every frame in order to display memory and performance statistics. If your game does not call Platform::RenderThread::drawFrame, you need to make sure AMP is updated every frame, by calling AmpServer::GetInstance().AdvanceFrame

4. **Why is AMP pausing one second or more to update (AMP::Server::WaitForAmpConnection)?**

Normally, AMP server is paused for a second if it consumes too much memory, so that any accumulated data may be sent to the client. If this continues to occur, it is possible that your content results in so much profiling information that AMP does not have time to send it all to the client without pausing Scaleform.

You can reduce the amount of profiling information being sent, by toggling off source line timing and detailed memory reports (the 'i' button on AMP client). There is also a 'profile level' dropdown in the AMP client, which can be lowered as well. Finally, you can increase the memory threshold that

triggers the problem. The default amount is set to 1MB, but you could modify it by calling GFx::AMP::Server::SetHeapLimit.

5. How much memory overhead can we expect when using the profiler?

By default, the AMP heap limit is 1MB, although this is not actually a hard limit, so it can be exceeded by a little. You can change this limit in your game by calling AmpServer::SetHeapLimit. The greater the limit, the less likely it will be that GFx will have to be paused while data is being sent to the AMP client.

7 Additional Information

For more information on AMP, please note the following resources:

- AMP Use Cases document.
- Scaleform Reference document – for details on the code/classes.
- AMP Forum – for community questions and support.